

Information Processing 1 — Week 12 exercises

Starting this week you have to complete short projects instead of exercises. Projects can be submitted the same week or the following week with no penalty. Projects that are completed, with challenges, and submitted the same week before the end of the exercises class will earn bonus points.

New projects will be added each week. Each project is worth a certain number of points. You can complete as many projects as you like. You can score up to 50 points in total from all the projects you complete. I recommend you complete as many projects as you can; do not stop when you reach 50 points.

When you finish a project, **immediately** ask an instructor to grade it. **Do not** struggle for more than a few minutes with anything you cannot solve. If you are stuck or have any questions then **raise your hand** and an instructor will help you.

1 Printing random words [5 points]

Task: print 20 random words from a novel. What are the steps needed, as a pseudo-code algorithm?

- read the words of the novel into a list [1.1]
- repeat 20 times:
 - pick a word randomly from the list [1.2]
 - print that word

1.1 Making a list of words from the contents of a file

You can write a reusable function called `fileWords(fileName)`. The pseudo-code for that function might look something like this:

function `fileWords(fileName)` is

- let `allWords` be an empty list
- let `file` be `fileName` opened for reading
- for each line in `file`:
 - split the line into individual words
 - add all those words to `allWords`
- return `allWords`

You already know how to do everything needed:

- `'def fileWords(fileName) :'` begins the function definition
- `'file = open(fileName)'` opens `fileName` for reading
- `'allWords = []'` creates an empty list to hold the result
- `'for line in file:'` iterates over each line in the open `file`
- `'words = line.split()'` splits `line` into individual words
- `'allWords.extend(words)'` extends `allWords` with the current line of `words`
- `'return allWords'` returns all the words in the file as the result of the function

You should be able to put all these together easily to write the function `fileWords(fileName)`.

When your function is written you should be able to write

```
words = fileWords("text3.txt")
```

to read the words from `text3.txt` into a list.

1.2 Choosing words at random from a list

You now have a list of all the words in a file. To pick words at random from that list there is a useful function in the `random` module called `random.choice(s)` which randomly chooses and then returns one of the elements in the sequence `s`. All you have to do then is `print()` it.

To print 20 words at random from the file, `print()` a `random.choice(words)` inside a loop that repeats 20 times.

Test your program by printing 20 random words from the file `text3.txt` which contains the novel "Emma" by Jane Austen.

▷▷ Ask an instructor to check and record your work now (before you run out of time).

2 Markov analysis [7 points]

Let's write a program to construct 'random' sentences, based on existing text, that look much more realistic than simple random words.¹

Random words do not look anything like an English sentence because there is no relationship between successive words. Markov analysis finds relationships between successive words. For each pair of consecutive words in some text it tells you what word(s) appeared next in the original text.

To print more realistic sentences, first create a Markov model of the sample text [2.1] and then print a sequence of words that are allowed by the model [2.2].

2.1 Building a Markov model of some text

The pseudo-code for building a Markov model of a list of words in a file might be something like this:

- let `markov` be a dictionary
- for each sequence of three words `a b c` in the file:
 - make a tuple `(a, b)`
 - ensure `markov[(a, b)]` is a list (initially empty)
 - append `c` to the list stored in `markov[(a, b)]`

At the end of this process, for any two words `v` and `w`, `markov[(v, w)]` gives you a list of words that are allowed to follow `v` and `w` in a sentence.

Previous things you have learned that might help include:

- `'markov = {}'` creates an empty dictionary

¹Perhaps you have heard of the AI system called 'Chat GPT'? Our program will be similar, albeit slightly simplified.

- `'for i in range(len(words)):'` iterates over the indexes of the list `words` (but be careful because you should stop just *before* the end, so that picking the three words *a*, *b*, and *c* does not use an illegal index beyond the end of the list)
- `'prefix = tuple(words[i:i+2])'` will slice two words from `words` starting at index *i*, returning them as a tuple
- `'suffix = words[i+2]'` will return the word in `words` that follows the two-word tuple
- `'markov.get(prefix, [])'` returns the entry for `prefix` in a dictionary, or an empty list if it is missing from the dictionary
- `'markov[prefix] = markov.get(prefix, []) + [suffix]'` will append `suffix` to the list value stored at `words[prefix]`, initialising it properly with an empty list if it does not already exist.

You can test your Markov model using the file `text4.txt` which contains a short nursery rhyme. After creating your model, use

```
for key, value in markov.items(): print(key, "->", value)
```

to print it. Using `text4.txt` your model should look like this:

```
('humpty', 'dumpty') -> ['sat', 'had']
('dumpty', 'sat') -> ['on']
('sat', 'on') -> ['a']
('on', 'a') -> ['wall']
('a', 'wall') -> ['humpty']
('wall', 'humpty') -> ['dumpty']
('dumpty', 'had') -> ['a']
('had', 'a') -> ['great']
('a', 'great') -> ['fall']
('great', 'fall') -> ['all']
('fall', 'all') -> ['the']
('all', 'the') -> ["king's", "king's"]
('the', "king's") -> ['horses', 'men']
("king's", 'horses') -> ['and']
('horses', 'and') -> ['all']
('and', 'all') -> ['the']
("king's", 'men') -> ["couldn't"]
('men', "couldn't") -> ['put']
("couldn't", 'put') -> ['humpty']
('put', 'humpty') -> ['together']
('humpty', 'together') -> ['again']
```

2.2 Printing sentences using a Markov model

The pseudo-code to print 'random' sentences might look like this:

- pick a random key from the Markov model
- print the words in that key (which is a tuple)
- repeat 20 times:

- lookup the list of words that can follow that key in the Markov model
- randomly pick one of those words
- print that word
- make a new key by combining
 - * the key with its first word removed
 - * the word you just randomly chose and printed above

Previous things you have learned that might help include:

- `'dict.keys()'` returns all the keys in a dictionary
- `'list(markov.keys())'` will make a list of all the keys in your Markov model dictionary
- `'random.choice(list(markov.keys()))'` will pick one of those keys at random
- `'" ".join(aTuple)'` will create a string containing each word in `aTuple` separated by a space
- `'print(aString, end="")'` will print `aString` without a newline following it
- `'words[prefix]'` is a list of words that can follow `prefix`
- `'suffix = random.choice(aList)'` will return a random element from `aList`
- `'aTuple[1:]'` will return a copy of `aTuple` with the first element removed
- `'aTuple[1:] + (suffix,)'` will add the element `suffix` to the end of that tuple

Test your program using `text4.txt` as your sample data. Your program should print more or less the original nursery rhyme.

To print a much more random sentence, use the file `text3.txt` as your 'training' data instead.

▷▷ Ask an instructor to check and record your work **now** (before you run out of time).

2.3 Challenge [bonus point]

Extend your Markov model program as follows:

- Begin the printed sentence with a capital letter, by repeatedly choosing a key until you find one whose first word begins with a capital letter. You might find the string method `str.isupper()` useful for this.
- Make sure the printed sentence ends with a full stop '.', by checking each word that you print for a full stop at the end. When you print one that does, `break` out of the loop to finish your sentence.
- Instead of using a fixed prefix length of 2, make the prefix length be variable. Define a variable at the start of your program called `prefixlen` and use it to configure the behaviour of your program.

▷▷ Ask an instructor to check and record your work **now** (before you run out of time).